The 5 Most Common Web Accessibility Challenges (And Their Solutions)

By Ryan Scott — Senior Accessibility Engineer (RScott Sites)

Ensuring web application accessibility means providing an equitable experience for all users, regardless of whether they navigate via mouse, keyboard, or assistive technologies like screen readers. During development, certain common implementation gaps can create significant barriers. The following sections outline five frequent accessibility challenges and the professional standards for resolving them.

1. Modal Dialogs and Focus Management

- **The Challenge:** When a modal dialog or pop-up appears, keyboard users may find that their focus remains on the underlying page content. This allows the cursor to "escape" the modal, leading to interactions with hidden elements and a confusing user experience.
- **Best Practices:** Developers should implement a "focus trap" within the modal. This ensures that keyboard navigation remains contained within the dialog until it is dismissed. Upon closing, focus must be programmatically returned to the original trigger element to maintain the user's context.

2. Semantic HTML and Button Components

- **The Challenge:** It is common for developers to style generic elements, such as generic text containers, to visually resemble buttons. However, these elements lack the native functionality required for keyboard interaction, failing to respond to standard 'Enter' or 'Space' key presses.
- **Best Practices:** Use native HTML button elements whenever possible. These elements provide built-in accessibility features, including proper role identification and keyboard event handling, which are automatically recognized by assistive technologies.

3. Dynamic Content and ARIA Live Regions

- **The Challenge:** Modern web applications often update content dynamically—such as displaying a "Success" notification—without a full page reload. While these updates are visible to sighted users, screen readers may not detect the change unless specifically instructed to do so.
- **Best Practices:** Utilize ARIA Live Regions to announce updates to screen reader users. By marking a container as a live region, the browser will automatically notify the assistive

technology when the content inside changes, ensuring all users are informed of system status updates.

4. Accessible Naming for Icon-Only Buttons

- **The Challenge:** Buttons that rely solely on icons, such as a trash can for deletion, provide visual context but often lack a programmatic label. Without a text alternative, a screen reader may only announce the element as a "button," leaving the user without an understanding of its purpose.
- **Best Practices:** Provide descriptive labels using attributes like aria-label (e.g., "Delete item"). This ensures that the function of the button is clearly communicated to screen reader users, providing the necessary clarity for confident navigation.

5. Input Validation and Error Identification

- **The Challenge:** When form validation fails, error messages are often displayed visually near the relevant input field. If these errors are not programmatically linked to the input, a screen reader user may be unaware that an error has occurred or which specific field requires correction.
- **Best Practices:** Use ARIA attributes like aria-describedby to create a digital association between the input field and its corresponding error message. This ensures that the error is announced as soon as the user focuses on the field, allowing for efficient troubleshooting and form submission.

Professional Accessibility Services

If your team needs a hands-on audit, developer training, or help fixing code, get in touch with **Ryan Scott (RScott Sites)**!

- **Website:** https://rscottsites.com
- **Contact:** ryanscott@rscottsites.com